EC Protocol

The **External Connections (EC) protocol** is the binary protocol that `amuled`, `amulegui`, `amuleweb`, and `amulecmd` use to communicate. It is a structured, efficient, and extensible binary protocol.

The protocol is defined in the aMule source under `src/libs/ec/` and `src/ExternalConn.cpp` — those files are the canonical reference if anything on this page conflicts with them.

EC packets carry data as nested, typed tags. The same tag-encoding idea (a type byte, a name, and a type-dependent value) underlies aMule's on-disk binary files; see File Formats for those layouts.

> **NOTE**
>
> EC is considered stable. Opcodes, tag names, and tag content formats can still change between releases. If you are implementing an EC client, pin the `EC_TAG_PROTOCOL_VERSION` to the version you have tested against.

Protocol Overview

EC is a two-layer protocol:

1. **Transmission layer** — an 8-byte header (4-byte flags + 4-byte packet body length) that precedes every application-layer packet.
2. **Application layer** — a tree-structured packet format with an opcode and nested tags.

All values in both layers are in **network byte order (big-endian, MSB first)**.

Section 1 — Transmission Layer

The transmission layer header is always exactly **8 bytes**: a 4-byte flags word followed by a 4-byte packet body length, both in big-endian (network) byte order.

```
[uint32]  flags         — capability and encoding flags (big-endian)
[uint32]  body_length   — byte count of the application-layer data that
follows
```

`body_length` does not include the 8-byte header itself.

Bit Definitions

Bit(s)	Name	Meaning
`0`	`EC_FLAG_ZLIB`	When set, the application-layer data is zlib-compressed.
`1`	`EC_FLAG_UTF8_NUMBERS`	When set, all integers in the application layer are encoded as UTF-8 wide characters (avoids zero bytes in small packets).
`2`	Reserved	Must be 0.
`3`	Reserved	Must be 0.
`4`	`EC_FLAG_LARGE_TAG_COUNT`	When set, `TAGCOUNT` fields use the sentinel-extended encoding (see Section 1.2). Both sides must have advertised `EC_TAG_CAN_LARGE_TAG_COUNT` in their authentication packet.
`5`	Always 1	Fixed protocol marker; acts as a sanity check.
`6`	Always 0	Fixed protocol marker; acts as a sanity check.
`7–31`	Reserved	Must be 0.

The sender always sets bit 5 and clears bit 6. The receiver rejects any packet where `(flags & 0x60) != 0x20`.

> **NOTE**

> Unlike `EC_FLAG_ZLIB` / `EC_FLAG_UTF8_NUMBERS` / `EC_FLAG_LARGE_TAG_COUNT`, the partial INC_UPDATE capability (`EC_TAG_CAN_PARTIAL_UPDATE`, see Section 3) has no transmission-layer flag bit. Once advertised by the client and echoed by the server at auth time it is active for the whole connection, with no per-packet toggle.

Example

```
00 00 00 20  00 00 00 43  <appdata>
```

Flags = `0x00000020` (bit 5 set, no compression, no UTF-8 numbers). Body length = 67 bytes (`0x43`).

Section 1.2 — Application Layer

The application layer consists of **packets** and **tags**.

Packet Structure

A packet is a special root tag — it has no data of its own, no `TAGLEN` field, and always has a `TAGCOUNT` field. All numbers in the application layer are big-endian (MSB first).

```
[ec_opcode_t]   OPCODE         // uint8: what the packet contains
[uint16]        TAGCOUNT       // number of first-level tags
<[uint32]       EXTENDED_TAGCOUNT>?  // only when
EC_FLAG_LARGE_TAG_COUNT
                <tags>
```

When `EC_FLAG_LARGE_TAG_COUNT` is in effect and `TAGCOUNT == 0xFFFF`, a `uint32` follows with the actual count. This sentinel-extended encoding lifts the historical 65535-tag ceiling for large responses (e.g. a shared-file list with more than 65535 entries).

Tag Structure

```
[ec_tagname_t]  TAGNAME        // uint16
[ec_tagtype_t]  TAGTYPE        // uint8
[ec_taglen_t]   TAGLEN         // uint32: total tag length including
sub-tags,
```

```
                                //          excluding
TAGNAME/TAGTYPE/TAGLEN fields
<[uint16]       TAGCOUNT>?    // only present if the tag has sub-tags
                <sub-tags>
                <tag data>
```

- `TAGNAME` on the wire is `(actual_code << 1) | has_children`. To recover the true tag name, right-shift by 1 (`actual_code = TAGNAME >> 1`). The lowest bit signals whether a `TAGCOUNT` and sub-tags are present.
- `TAGTYPE` identifies the data type (see `ECTagTypes.h` in the source).
- `TAGLEN` includes the length of all sub-tags (plus their `TAGCOUNT` field) but excludes the header fields (`TAGNAME`, `TAGTYPE`, `TAGLEN`) themselves.
- When a tag has sub-tags, they appear **before** the tag's own data. Own data length = `TAGLEN − (sum of all sub-tag total lengths) − sizeof(TAGCOUNT)`.
- When `EC_FLAG_LARGE_TAG_COUNT` is in effect, the sub-tag `TAGCOUNT` uses the same sentinel-extended encoding as the packet-level `TAGCOUNT`.

Example: a tag with `TAGNAME = 0x0E03` on the wire has `has_children = 1` (bit 0 set) and true tag code = `0x0E03 >> 1 = 0x0701` = `EC_TAG_SEARCH_TYPE`.

Section 2 — Data Types

Integer Types

All integer types (`uint8`, `uint16`, `uint32`, `uint64`) are in network byte order (big-endian) in the application layer.

aMule encodes each integer with the **narrowest type that fits its value**: `CECTag::InitInt` (`ECTag.cpp`) picks `UINT8` for values `≤ 0xFF`, `UINT16` for `≤ 0xFFFF`, `UINT32` for `≤ 0xFFFFFFFF`, otherwise `UINT64` — regardless of the C++ width the sender used. So a field documented as `uint32` may arrive on the wire as `UINT8`, `UINT16`, or `UINT32` depending on the value. Clients **must** read integers with a width-agnostic reader (e.g. `GetInt()`) and never assume the declared width. This is why the `EC_TAG_PROTOCOL_VERSION` example below is `UINT16` even though the sender passes it as a 64-bit value.

When `EC_FLAG_UTF8_NUMBERS` is set, integers are encoded as UTF-8 wide characters to avoid zero bytes. Consumers can pass them through a UTF-8 decoder to obtain the original value.

Strings

Strings are always **UTF-8**, including the trailing zero byte. All strings from the server are untranslated; their translations are in `amule.mo`.

Boolean

- **When reading**: presence of the tag means `true`; absence means `false`.
- **When writing**: the tag must always be present and hold a `uint8` (0 = false, non-zero = true). If the tag is absent on write, the receiver treats it as *unchanged*.

Boolean values are primarily used for reading and writing preferences.

MD4/MD5 Hashes

Always 16 raw bytes (byte order does not apply to the hash itself), stored with `EC_TAGTYPE_HASH16`.

IPv4 Addresses

An IPv4 address+port is packed as 6 bytes: 4 bytes for the IP (network order) followed by 2 bytes for the port (network order).

Floating-Point Numbers

`float` and `double` values are converted to their string representation and sent as strings. The decimal separator is always `.` (dot), regardless of locale.

Section 3 — Authentication

Authentication is a **three-step** challenge-response exchange at the start of every session. The client identifies itself and advertises capabilities; the server issues a random salt; the client hashes the password with the salt and sends the result.

Step 1 — Auth Request (client → server)

```
EC_OP_AUTH_REQ (0x02)
```

```
    +-- EC_TAG_CLIENT_NAME         (optional) — name of the EC client
application
    +-- EC_TAG_CLIENT_VERSION      (optional) — version of the EC
client application
    +-- EC_TAG_PROTOCOL_VERSION    (required) — EC protocol version
(currently 0x0204)
    +-- EC_TAG_VERSION_ID          (dev builds only, must NOT appear in
releases) — MD5 hash of `ECCodes.h`
    +-- EC_TAG_CAN_ZLIB            (optional) — advertises zlib
compression support
    +-- EC_TAG_CAN_UTF8_NUMBERS    (optional) — advertises UTF-8 number
encoding support
    +-- EC_TAG_CAN_NOTIFY          (optional) — advertises notification
support
    +-- EC_TAG_CAN_LARGE_TAG_COUNT (optional) — advertises sentinel-
extended tag count support
    +-- EC_TAG_CAN_PARTIAL_UPDATE  (optional) — advertises partial
INC_UPDATE support
    +-- EC_TAG_PREFER_NO_ZLIB      (optional) — hint: skip per-packet
zlib (client dialed a loopback/LAN IP)
```

Each `EC_TAG_CAN_*` is an empty tag (type `CUSTOM`, zero-length data) advertising a capability. `EC_TAG_PASSWD_HASH` does **not** appear in this packet.

`EC_TAG_VERSION_ID` is the **MD5** digest of `ECCodes.h` (with comments and whitespace normalised, so only real code changes alter it). It is transported in a 16-byte `HASH16` container, but the algorithm is MD5, not MD4. It is compiled in only on development builds (gated by the `EC_VERSION_ID` macro in `src/ExternalConn.cpp`) and must never appear in release builds.

Step 2 — Salt (server → client)

```
EC_OP_AUTH_SALT (0x4F)
    +-- EC_TAG_PASSWD_SALT  (required) — uint64 random salt value
```

Step 3 — Password (client → server)

The client computes the salted hash:

```
salt_hex   = upper-hex-string(salt_value)           // e.g.
"5E3AB49C174F0C02"
salt_hash  = MD5(salt_hex)                          // hex-encoded MD5
of the hex salt
```

```
final_hash = MD5(lower(MD5(password)) + salt_hash)   // MD5 of
concatenated hex strings
```

`salt_hex` is produced with a `"%lX"` format (`ExternalConn.cpp`): **uppercase** hex, **no leading-zero padding**, and **no `0x` prefix**. The salt is a full random `uint64`, so it is usually 16 hex digits, but a value whose high nibble is zero yields fewer digits — do not zero-pad to 16. `MD5(password)` is the password already stored as a lowercase MD5 hex string in aMule's preferences.

Then sends:

```
EC_OP_AUTH_PASSWD (0x50)
    +-- EC_TAG_PASSWD_HASH  (required) — HASH16: 16-byte salted MD5
result above
```

Step 4 — Server Reply

On success:

```
EC_OP_AUTH_OK (0x04)
    +-- EC_TAG_SERVER_VERSION      (always)   — aMule version string
    +-- EC_TAG_CAN_LARGE_TAG_COUNT (optional) — echoed if the server
accepts the feature
    +-- EC_TAG_CAN_PARTIAL_UPDATE  (optional) — echoed if partial
INC_UPDATE mode is active
```

The server echoes only the capabilities it will honour. The client must not use a capability unless the server echoed it.

On failure:

```
EC_OP_AUTH_FAIL (0x03)
    +-- EC_TAG_STRING  (optional) — human-readable reason
```

Wire-Level Example

Step 1 — Auth request (8 tags, no compression):

```
00 00 00 20                          FLAGS: bit 5 set, no compression
00 00 00 4a                          Body length: 74 bytes
02                                   EC_OP_AUTH_REQ
  00 08                              Tag count: 8
    02 00                            EC_TAG_CLIENT_NAME (0x0100, no
children)
      06                             EC_TAGTYPE_STRING
      00 00 00 09                    TagLen: 9
      61 4d 75 6c 65 63 6d 64 00   "aMulecmd\0"
    02 02                            EC_TAG_CLIENT_VERSION (0x0101, no
children)
      06                             EC_TAGTYPE_STRING
      00 00 00 04                    TagLen: 4
      43 56 53 00                    "CVS\0"
    00 04                            EC_TAG_PROTOCOL_VERSION (0x0002, no
children)
      03                             EC_TAGTYPE_UINT16
      00 00 00 02                    TagLen: 2
      02 04                          0x0204 (current protocol version)
    00 18                            EC_TAG_CAN_ZLIB (0x000C, no
children)
      01                             EC_TAGTYPE_CUSTOM
      00 00 00 00                    TagLen: 0
    00 1a                            EC_TAG_CAN_UTF8_NUMBERS (0x000D, no
children)
      01                             EC_TAGTYPE_CUSTOM
      00 00 00 00                    TagLen: 0
    00 1c                            EC_TAG_CAN_NOTIFY (0x000E, no
children)
      01                             EC_TAGTYPE_CUSTOM
      00 00 00 00                    TagLen: 0
    00 22                            EC_TAG_CAN_LARGE_TAG_COUNT (0x0011,
no children)
      01                             EC_TAGTYPE_CUSTOM
      00 00 00 00                    TagLen: 0
    00 24                            EC_TAG_CAN_PARTIAL_UPDATE (0x0012,
no children)
      01                             EC_TAGTYPE_CUSTOM
      00 00 00 00                    TagLen: 0
```

Step 2 — Salt (server → client):

```
00 00 00 20                          FLAGS
00 00 00 12                          Body length: 18 bytes
4f                                   EC_OP_AUTH_SALT
```

```
  00 01                          Tag count: 1
    00 16                        EC_TAG_PASSWD_SALT (0x000B, no
children)
      05                         EC_TAGTYPE_UINT64
      00 00 00 08                TagLen: 8
      5e 3a b4 9c 17 4f 0c 02    salt value (example)
```

Step 3 — Password (client → server):

```
00 00 00 20                      FLAGS
00 00 00 1a                      Body length: 26 bytes
50                               EC_OP_AUTH_PASSWD
  00 01                          Tag count: 1
    00 02                        EC_TAG_PASSWD_HASH (0x0001, no
children)
      09                         EC_TAGTYPE_HASH16
      00 00 00 10                TagLen: 16
      5d 41 40 2a bc 4b 2a 76    salted MD5 hash of password
      b9 71 9d 91 10 17 c5 92
```

Step 4 — Auth OK (server → client):

```
00 00 00 20                      FLAGS
00 00 00 15                      Body length: 21 bytes
04                               EC_OP_AUTH_OK
  00 02                          Tag count: 2
    0a 16                        EC_TAG_SERVER_VERSION (0x050B, no
children)
      06                         EC_TAGTYPE_STRING
      00 00 00 04                TagLen: 4
      43 56 53 00                "CVS\0"
    00 22                        EC_TAG_CAN_LARGE_TAG_COUNT (0x0011,
no children)
      01                         EC_TAGTYPE_CUSTOM
      00 00 00 00                TagLen: 0
```

UTF-8 Encoded Tag Names

When `EC_FLAG_UTF8_NUMBERS` is active, tag names are also encoded as UTF-8 wide characters. The value encoded is `(actual_code << 1) | has_children`. Example:

```
c8 80
```

UTF-8 decode: `c8 80` → `0x0200`. Bit 0 = 0 (no children). True code = `0x0200 >> 1 = 0x0100` = `EC_TAG_CLIENT_NAME` (0x0100 in `ECCodes.h`).

Partial Updates (INC_UPDATE)

A client that advertises `EC_TAG_CAN_PARTIAL_UPDATE` (0x0012) in its auth request tells the server it understands the **partial incremental-update** protocol. When the server activates this mode it echoes `EC_TAG_CAN_PARTIAL_UPDATE` in `EC_OP_AUTH_OK`.

In a partial `EC_DETAIL_INC_UPDATE` response (e.g. the download or shared-file list), the server may then **omit files that have not changed** and signal removals explicitly with an `EC_TAG_FILE_REMOVED` (0x0013) tag, instead of the legacy "absence implies deletion" convention. The feature is backward-compatible: a server talking to a client that did **not** advertise the capability (or an old server that does not implement it) falls back to emitting an alive marker for every unchanged file, so the client's bulk "missing == deleted" logic still works.

ZLIB Locality Preference

`EC_TAG_CAN_ZLIB` (0x000C) is the **capability** (zlib stays available, and is always used for oversized payloads). `EC_TAG_PREFER_NO_ZLIB` (0x0014) is an optional **preference** asking the server to skip per-packet zlib for small/medium payloads, where it would be pure CPU overhead on a fast link.

The preference is set by the **client** — the only side that knows the IP it dialed — when the resolved server address is loopback, RFC1918 LAN or RFC3927 link-local. The user can override it (`amulegui`'s **Force ZLIB compression** checkbox, or `--force-zlib` / `/EC/ForceZLIB` for `amulecmd` and `amuleweb`), in which case the tag is not sent. An old server ignores the unknown tag and an old client never sends it; both fall back to always-zlib.

Section 4 — Common Operations

Stats Request

```
EC_OP_STAT_REQ (0x0a)
    +-- EC_TAG_DETAIL_LEVEL (EC_DETAIL_CMD = 0)
```

Wire encoding:

```
00 00 00 20                          FLAGS
00 00 00 09                          Body length: 9 bytes
0a                                   EC_OP_STAT_REQ
  00 01                              Tag count: 1
    00 08                            EC_TAG_DETAIL_LEVEL (0x0004, no
children)
      02                             EC_TAGTYPE_UINT8
      00 00 00 01                    TagLen: 1
      00                             0 = EC_DETAIL_CMD
```

Reply (`EC_DETAIL_CMD` level):

```
EC_OP_STATS (0x0c)
    +-- EC_TAG_STATS_UL_SPEED           (0x0200) uint32 — upload rate
(B/s)
    +-- EC_TAG_STATS_DL_SPEED           (0x0201) uint32 — download
rate (B/s)
    +-- EC_TAG_STATS_UL_SPEED_LIMIT     (0x0202) uint32 — upload limit
(B/s)
    +-- EC_TAG_STATS_DL_SPEED_LIMIT     (0x0203) uint32 — download
limit (B/s)
    +-- EC_TAG_STATS_UL_QUEUE_LEN       (0x0208) uint32 — waiting
upload queue length
    +-- EC_TAG_STATS_TOTAL_SRC_COUNT    (0x0206) uint32 — total found
sources
    +-- EC_TAG_STATS_ED2K_USERS         (0x0209) uint32 — users on
connected eD2k server
    +-- EC_TAG_STATS_KAD_USERS          (0x020A) uint32 — estimated
Kad users
    +-- EC_TAG_STATS_ED2K_FILES         (0x020B) uint32 — files on
connected eD2k server
    +-- EC_TAG_STATS_KAD_FILES          (0x020C) uint32 — estimated
Kad files
    +-- EC_TAG_STATS_KAD_NODES          (0x021B) uint32 — known Kad
nodes
```

The 11 tags above are the minimum reply when Kademlia is *not* connected. When `Kademlia::CKademlia::IsConnected()` returns true the reply additionally contains `EC_TAG_STATS_KAD_FIREWALLED_UDP`, `KAD_INDEXED_SOURCES`, `KAD_INDEXED_KEYWORDS`, `KAD_INDEXED_NOTES`, `KAD_INDEXED_LOAD`, `KAD_IP_ADDRESS`, `KAD_IN_LAN_MODE`, `BUDDY_STATUS`, `BUDDY_IP`, and `BUDDY_PORT`.

`EC_DETAIL_FULL` and `EC_DETAIL_INC_UPDATE` additionally prepend `STATS_UP_OVERHEAD`, `STATS_DOWN_OVERHEAD`, `STATS_BANNED_COUNT`, a logger tag, `STATS_TOTAL_SENT_BYTES`, `STATS_TOTAL_RECEIVED_BYTES`, and `STATS_SHARED_FILE_COUNT`. See `Get_EC_Response_StatRequest` in `ExternalConn.cpp` for the exact assembly, and `ECCodes.h` for the full `0x0200` tag range.

Partial wire decoding of the reply:

```
00 00 00 20                              FLAGS
00 00 00 5f                              Body length: 95 bytes
0c                                       EC_OP_STATS
  00 0b                                  First-level tag count: 11
    04 00 [...]                          EC_TAG_STATS_UL_SPEED (0x0200)
    04 02 [...]                          EC_TAG_STATS_DL_SPEED (0x0201)
    04 04 [...]                          EC_TAG_STATS_UL_SPEED_LIMIT
(0x0202)
    04 06 [...]                          EC_TAG_STATS_DL_SPEED_LIMIT
(0x0203)
    04 10 [...]                          EC_TAG_STATS_UL_QUEUE_LEN (0x0208)
    04 0c [...]                          EC_TAG_STATS_TOTAL_SRC_COUNT
(0x0206)
    04 12 [...]                          EC_TAG_STATS_ED2K_USERS (0x0209)
    04 14 [...]                          EC_TAG_STATS_KAD_USERS (0x020A)
    04 16 [...]                          EC_TAG_STATS_ED2K_FILES (0x020B)
    04 18 [...]                          EC_TAG_STATS_KAD_FILES (0x020C)
    04 36 [...]                          EC_TAG_STATS_KAD_NODES (0x021B)
```

Connection State

Connection state is requested separately with `EC_OP_GET_CONNSTATE` (0x0B). The reply uses opcode `EC_OP_MISC_DATA` (0x07) and contains `EC_TAG_CONNSTATE` with the current UserID as its own data and `EC_TAG_SERVER` as a child:

```
EC_OP_GET_CONNSTATE (0x0b)
    +-- EC_TAG_DETAIL_LEVEL
```

Reply:

```
EC_OP_MISC_DATA (0x07)
    +-- EC_TAG_CONNSTATE (0x0005)  — uint32 UserID; has children when
connected
```

```
        +-- EC_TAG_SERVER (0x0500) — IPv4: server IP+port; has children
            +-- EC_TAG_SERVER_NAME (0x0501) — string
```

Partial wire decoding:

```
00 00 00 20                          FLAGS
...
07                                   EC_OP_MISC_DATA
  00 01                              Tag count: 1
    00 0b                            EC_TAG_CONNSTATE (0x0005, has
children — wire = 0x000B)
      04                             EC_TAGTYPE_UINT32
      00 00 00 28                    TagLen: 40
      00 01                          Child tag count: 1
        0a 01                        EC_TAG_SERVER (0x0500, has children
— wire = 0x0A01)
          08                         EC_TAGTYPE_IPV4
          00 00 00 1b                TagLen: 27
          00 01                      Child tag count: 1
            0a 02                    EC_TAG_SERVER_NAME (0x0501, no
children — wire = 0x0A02)
              06                     EC_TAGTYPE_STRING
              00 00 00 0e            TagLen: 14
              52 61 7a 6f 72 62 61   "Razorback 2.0\0"
              63 6b 20 32 2e 30 00
          c3 f5 f4 f3 12 35          EC_TAG_SERVER data: IP
195.245.244.243, port 4661
      90 cc 83 52                    EC_TAG_CONNSTATE data: current
UserID
```

How the `TAGLEN` values arise (matching `CECTag::GetTagLen` in `ECTag.cpp`):

- `EC_TAG_SERVER_NAME`: 14 (own data) → `TAGLEN = 14`.
- `EC_TAG_SERVER`: 6 (own IPv4 data) + 7 (`SERVER_NAME` header: `TAGNAME` + `TAGTYPE` + `TAGLEN`) + 14 (`SERVER_NAME` data) → `TAGLEN = 27`. Note that `SERVER`'s own `TAGCOUNT` field is *not* counted in its own `TAGLEN` — only each sub-tag's contribution is.
- `EC_TAG_CONNSTATE`: 4 (own UserID data) + 7 (`SERVER` header) + 2 (`SERVER`'s `TAGCOUNT`, because `SERVER` has children) + 27 (`SERVER`'s `TAGLEN`) → `TAGLEN = 40`.

Search Request

```
EC_OP_SEARCH_START (0x26)
```

```
    +-- EC_TAG_SEARCH_TYPE
        +-- EC_TAG_SEARCH_NAME
        +-- EC_TAG_SEARCH_FILE_TYPE
```

Wire encoding (plain format, no compression):

```
00 00 00 20                          FLAGS
00 00 00 21                          Packet body length: 33 bytes
26                                   EC_OP_SEARCH_START
  00 01                              Tag count: 1
    0e 03                            EC_TAG_SEARCH_TYPE (0x0701, has
children — wire = 0x0E03)
      02                             EC_TAGTYPE_UINT8
      00 00 00 17                    TagLen: 23
      00 02                          Child tag count: 2
        0e 04                        EC_TAG_SEARCH_NAME (0x0702, no
children — wire = 0x0E04)
          06                         EC_TAGTYPE_STRING
          00 00 00 05                TagLen: 5
          74 65 73 74 00             "test\0"
        0e 0a                        EC_TAG_SEARCH_FILE_TYPE (0x0705, no
children — wire = 0x0E0A)
          06                         EC_TAGTYPE_STRING
          00 00 00 01                TagLen: 1
          00                         "\0" (empty = any type)
      00                             uint8 search type: 0 =
EC_SEARCH_LOCAL
```

Section 5 — Preferences Tags

Connection Preferences (`EC_TAG_PREFS_CONNECTIONS = 0x1300`)

Tag	Code	Type	Description
`EC_TAG_CONN_DL_CAP`	`0x1301`	`uint32`	Download line capacity (kB/s)
`EC_TAG_CONN_UL_CAP`	`0x1302`	`uint32`	Upload line capacity (kB/s)

Tag	Code	Type	Description
`EC_TAG_CONN_MAX_DL`	`0x1303`	`uint32`	Max download speed (kB/s)
`EC_TAG_CONN_MAX_UL`	`0x1304`	`uint32`	Max upload speed (kB/s)
`EC_TAG_CONN_SLOT_ALLOCATION`	`0x1305`	`uint32`	Upload slot allocation
`EC_TAG_CONN_MAX_FILE_SOURCES`	`0x1309`	`uint16`	Max sources per file
`EC_TAG_CONN_MAX_CONN`	`0x130A`	`uint16`	Max connections

> **NOTE**
>
> `EC_TAG_CONN_MAX_DL`, `EC_TAG_CONN_MAX_UL`, and `EC_TAG_CONN_SLOT_ALLOCATION` were widened from `uint16` to `uint32` to support speeds above 65534 kB/s (~524 Mbps) needed on modern gigabit connections. EC clients reading these tags should use `GetInt()` (which handles any integer width); clients sending them should encode them as 32-bit values.

Section 6 — Implementing an EC Client

EC can be used to build custom clients for aMule in any language. A minimal client needs to:

1. Open a TCP connection to `amuled` (default port 4712 — the port, password, and whether EC is enabled at all are set in the `[ExternalConnect]` section of `amule.conf`).
2. Send the 8-byte transmission header followed by `EC_OP_AUTH_REQ` with `EC_TAG_PROTOCOL_VERSION` and capability tags.
3. Receive `EC_OP_AUTH_SALT` and extract the `EC_TAG_PASSWD_SALT` value (uint64).
4. Compute `final_hash = MD5(lower(MD5(password)) + MD5(upper_hex(salt)))`, where `upper_hex(salt)` is the salt formatted as **uppercase** hex with no leading-zero padding and no `0x` prefix.
5. Send `EC_OP_AUTH_PASSWD` with `EC_TAG_PASSWD_HASH` = the computed hash.
6. Receive the reply. If `EC_OP_AUTH_OK`, proceed; if `EC_OP_AUTH_FAIL`, the password was wrong or another error occurred.
7. Send request packets and receive reply packets in a request/response loop.

Minimal flags value: Use `0x00000020` (32-bit big-endian) as the flags word. aMule will not use any extensions if you do not advertise them.

Reference Implementations

The canonical implementation is in the aMule source:

- `src/libs/ec/cpp/` — C++ client library (shared by `amulegui`, `amulecmd`)
- `src/ExternalConn.cpp` — server-side implementation in `amuled`
- `src/libs/ec/cpp/ECCodes.h` — all opcode and tag name constants
- `src/libs/ec/cpp/ECTagTypes.h` — tag type constants
- `src/libs/ec/cpp/ECTag.cpp` — tag wire encoding/decoding (see `WriteTag`, `ReadFromSocket`)
- `src/libs/ec/cpp/ECSocket.cpp` — transmission layer (see `WritePacket`, `ReadHeader`)

Section 7 — Design Notes

Key design goals of the protocol:

- **Security** — salted MD5 password hashing; extensible capability negotiation.
- **Efficiency** — zlib compression for large responses; UTF-8 integer encoding for small packets.
- **Flexibility** — nested tag structure allows adding new fields without breaking existing clients.

The protocol can be modelled as binary XML, or equivalently as a tree: one root (the packet), with any number of branches (tags) and leaves (tags with no children).